基础
- CPP/STL
- 结论

贪心

数学
- 组合数学
  - 一、基本计数原理
  - 二、组合恒等式
  - 三、容斥原理
  - 四、卡特兰数
  - 五、斯特林数
  - 六、贝尔数
  - 七、整数拆分
  - 八、生成函数
  - 九、鸽巢原理
  - 十、卢卡斯定理与扩展卢卡斯
  - 十一、其他常用结论
- 线性代数
  - 一、矩阵基础
  - 二、行列式
  - 三、高斯消元
  - 四、矩阵求逆
  - 五、线性基（异或空间）
  - 六、特征值与特征向量
  - 七、矩阵与递推
  - 八、特殊矩阵
  - 九、线性代数在竞赛中的其他应用
  - 十、Berlekamp-Massey 算法
- 积分
  - 积分结论
- 数论函数
  - 一、积性函数与完全积性函数
  - 二、欧拉函数 $\varphi(n)$
  - 三、莫比乌斯函数 $\mu(n)$
  - 四、约数个数 $d(n)$ 与约数和 $\sigma(n)$
  - 五、狄利克雷卷积
  - 六、莫比乌斯反演
  - 七、杜教筛
  - 八、min_25 筛
  - 九、洲阁筛
  - 十、常见积性函数
- 数论
  - 常见数字
  - 二次剩余
  - 数域扩充

图论
- 各种图
  - 竞赛图
- 线段树优化建图
- 长链剖分

数据结构
- 线段树
- 树状数组
- CDQ分治

杂项
- 卡常

代码审查

基础

CPP/STL

重载 set：

- **使用仿函数**，顺序与排序顺序一致；
- **比较器必须严格弱序**，返回 `true` 表示 `a` 应排在 `b` 前面”。`<=`、`>=` 都会导致 UB。
- **比较器本身要** `const`，即 `operator()` 必须 `const`，否则某些 STL 实现会编译报错。
- **比较器不能依赖运行时可变的全局状态**，否则破坏 `set` 的不变式。
- `set` **判断相等是靠** `!(a<b) && !(b<a)`，不是 `==`。如果你只按 `x` 排序，那么 `x` 相同的元素会被认为是同一个，插入会被忽略。要保留多个，用 `multiset` 或把 `y` 也纳入比较。

std::print

- 需要 cpp 23；
- 打印二进制：`std::print("{:b}", x);`

结论

1. a 的无限循环字符串和 b 的无限循环字符串比较字典序：只需比较前 $|a| + |b| - \gcd(a, b)$ 个字符即可，实际上 $a^\infty < b^\infty \Leftrightarrow a + b < b + a$，其他同理；

贪心

- 给定数组，每次操作可以选一个数加一或者减一，最终要使所有数互不相同，求最小操作次数：

  先将数组排序，令 $b_i = a_i - i$，原问题变为使数组单调不减，直接用堆维护即可。

```cpp
#include <bits/stdc++.h>
using namespace std;

long long minOperations(vector<int>& a) {
    sort(a.begin(), a.end());

    priority_queue<long long> pq; // 最大堆
    long long ans = 0;

    for (int i = 0; i < a.size(); i++) {
        long long b = (long long)a[i] - i;

        pq.push(b);

        if (pq.top() > b) {
            ans += pq.top() - b;
            pq.pop();
            pq.push(b);
        }
    }

    return ans;
}
```

- 给定一个字符串，重排它使得没有相邻相同的字符。选取最大的出现次数 mx，如果 $mx \leq \lceil n/2 \rceil$ 则有解，否则无解；构造：每次取剩余次数最大且不等于上一个字符的字符；
- 给定序列 $\{a\}$，每次允许选择一个数 $\div 2(\text{floor}), +1, -1$，求使所有序列相同的最小操作数：结论，答案一定在某个塔的 $\div 2$ 链上，这个结论对于只有 $+1$ 和只有 -1 都是对的；

数学

组合数学

一、基本计数原理

1. 加法原理与乘法原理

- 加法原理：若方案分为互不相交的若干类，第 i 类有 a_i 种，则总方案数为 $\sum a_i$。
- 乘法原理：若方案分步进行，第 i 步有 a_i 种，则总方案数为 $\prod a_i$。

2. 排列与组合

- 排列数：

$$A_n^m = P_n^m = \frac{n!}{(n-m)!}$$

- 组合数：

$$\binom{n}{m} = C_n^m = \frac{n!}{m!(n-m)!}$$

- 可重排列：n 种元素取 m 个（可重复）的排列数为 n^m。
- 可重组合：n 种元素取 m 个（可重复）的组合数为

$$\binom{n+m-1}{m}$$

- 不全相异元素排列：若 n 个元素中有 n_1 个相同元素 1，n_2 个相同元素 2，…，n_k 个相同元素 k，且 $\sum n_i = n$，则排列数为

$$\frac{n!}{n_1!n_2!\cdots n_k!}$$

3. 圆排列与项链排列

- 圆排列：n 个不同元素围成一圈的排列数为

$$(n-1)!$$

- 项链排列（可翻转）：n 个不同元素的项链数为

$$\frac{(n-1)!}{2} \quad (n \geq 3)$$

二、组合恒等式

1. 对称恒等式

$$\binom{n}{m} = \binom{n}{n-m}$$

2. 吸收恒等式

$$\binom{n}{m} = \frac{n}{m}\binom{n-1}{m-1}$$

$$m\binom{n}{m} = n\binom{n-1}{m-1}$$

3. 加法公式（帕斯卡恒等式）

$$\binom{n}{m} = \binom{n-1}{m} + \binom{n-1}{m-1}$$

4. 范德蒙德卷积

$$\sum_{i=0}^{k} \binom{n}{i}\binom{m}{k-i} = \binom{n+m}{k}$$

更一般地：

$$\sum_{i} \binom{n}{i}\binom{m}{k-i} = \binom{n+m}{k}$$

5. 上指标求和

$$\sum_{i=0}^{n} \binom{i}{m} = \binom{n+1}{m+1}$$

$$\sum_{i=m}^{n} \binom{i}{m} = \binom{n+1}{m+1}$$

6. 交错和

$$\sum_{i=0}^{n} (-1)^i \binom{n}{i} = 0 \quad (n \geq 1)$$

$$\sum_{i=0}^{n} (-1)^i \binom{n}{i} a^i = (1-a)^n$$

7. 二项式定理

$$(x+y)^n = \sum_{i=0}^{n} \binom{n}{i} x^i y^{n-i}$$

特例：

$$\sum_{i=0}^{n} \binom{n}{i} = 2^n$$

$$\sum_{i=0}^{n} \binom{n}{i} (-1)^i = 0 \quad (n \geq 1)$$

$$\sum_{i=0}^{n} \binom{n}{i} i = n2^{n-1}$$

$$\sum_{i=0}^{n} \binom{n}{i} i^2 = n(n+1)2^{n-2}$$

8. 广义二项式定理

对于任意实数 α：

$$(1+x)^\alpha = \sum_{i=0}^{\infty} \binom{\alpha}{i} x^i$$

其中

$$\binom{\alpha}{i}=\frac{\alpha(\alpha-1)\cdots(\alpha-i+1)}{i!}$$

9. 组合数乘积恒等式

$$\binom{n}{m}\binom{m}{k}=\binom{n}{k}\binom{n-k}{m-k}$$

10. 二项式系数与幂次

$$\sum_{i=0}^{n}\binom{n}{i}^2=\binom{2n}{n}$$

$$\sum_{i=0}^{n}\binom{n}{i}\binom{n}{n-i}=\binom{2n}{n}$$

11. 多重集组合

从 n 种元素中取 m 个（每种至少取 0 个）：

$$\binom{n+m-1}{m}$$

若每种至少取 1 个：

$$\binom{m-1}{n-1}$$

12. 隔板法

把 m 个相同物品分给 n 个人（允许有人没有）：

$$\binom{m+n-1}{n-1}$$

每人至少一个：

$$\binom{m-1}{n-1}$$

13. 组合数与斐波那契数列

$$\sum_{y=0}^{x}\binom{x}{y}f(p+y)=f(p+2x)$$

其中 $f(x)$ 是斐波那契数列，x,p 已给定

14. 组合数递推

$$\binom{x+1}{y}=\binom{x}{y}\frac{x+1}{x+1-y}$$

$$\binom{x}{y+1}=\binom{x}{y}\frac{x-y}{y+1}$$

三、容斥原理

1. 基本形式

设 $A_1, A_2, \ldots, A_n$ 为有限集合，则

$$\left|\bigcup_{i=1}^{n} A_i\right| = \sum_{i=1}^{n} |A_i| - \sum_{1\le i<j\le n} |A_i \cap A_j| + \sum_{1\le i<j<k\le n} |A_i \cap A_j \cap A_k| - \cdots + (-1)^{n-1} |A_1 \cap A_2 \cap \cdots \cap A_n|$$

简写：

$$\left|\bigcup_{i=1}^{n} A_i\right| = \sum_{\emptyset \neq S \subseteq [n]} (-1)^{|S|-1} \left|\bigcap_{i\in S} A_i\right|$$

2. 补集形式

$$\left|\overline{A_1} \cap \overline{A_2} \cap \cdots \cap \overline{A_n}\right| = \sum_{S\subseteq[n]} (-1)^{|S|} \left|\bigcap_{i\in S} A_i\right|$$

其中 $S = \emptyset$ 时交集为全集。

3. 常见应用

- 错排问题：n 个元素的错排数

$$D_n = n! \sum_{i=0}^{n} \frac{(-1)^i}{i!}$$

递推：

$$D_n = (n-1)(D_{n-1} + D_{n-2}), \quad D_0 = 1, D_1 = 0$$

- 欧拉函数：

$$\varphi(n) = n \prod_{p|n} \left(1 - \frac{1}{p}\right)$$

- 求 $1 \sim n$ 中与 n 互质的数的个数即为 $\varphi(n)$。

四、卡特兰数

1. 定义

$$C_n = \frac{1}{n+1}\binom{2n}{n}$$

前几项：$C_0 = 1, C_1 = 1, C_2 = 2, C_3 = 5, C_4 = 14, C_5 = 42$。

2. 递推

$$C_n = \sum_{i=0}^{n-1} C_i C_{n-1-i}$$

$$C_n = \frac{2(2n-1)}{n+1} C_{n-1}$$

3. 常见模型

- n 对括号的合法括号序列数。
- n 个节点的不同二叉树形态数。
- 从 $(0,0)$ 到 (n,n) 不越过对角线 $y=x$ 的路径数。
- 凸 $n+2$ 边形的三角剖分数。
- n 个元素的出栈序列数。

4. 扩展卡特兰数

从 $(0,0)$ 到 (n,m) 且不越过 $y=x$ 的路径数（$n \geq m$）：

$$\binom{n+m}{m}-\binom{n+m}{m-1}$$

五、斯特林数

1. 第二类斯特林数

$S(n,m)$ 或 $\begin{Bmatrix} n \\ m \end{Bmatrix}$：把 n 个不同元素分成 m 个非空子集的方案数。

递推：

$$\begin{Bmatrix} n \\ m \end{Bmatrix}=\begin{Bmatrix} n-1 \\ m-1 \end{Bmatrix}+m\begin{Bmatrix} n-1 \\ m \end{Bmatrix}$$

通项：

$$\begin{Bmatrix} n \\ m \end{Bmatrix}=\frac{1}{m!}\sum_{i=0}^{m}(-1)^i\binom{m}{i}(m-i)^n$$

重要恒等式：

$$x^n=\sum_{m=0}^{n}\begin{Bmatrix} n \\ m \end{Bmatrix}x^{\underline{m}}$$

其中 $x^{\underline{m}}=x(x-1)\cdots(x-m+1)$ 为下降幂。

2. 第一类斯特林数

$\begin{bmatrix} n \\ m \end{bmatrix}$：把 n 个不同元素排成 m 个非空循环的方案数。

递推：

$$\begin{bmatrix} n \\ m \end{bmatrix}=\begin{bmatrix} n-1 \\ m-1 \end{bmatrix}+(n-1)\begin{bmatrix} n-1 \\ m \end{bmatrix}$$

重要恒等式：

$$x^{\overline{n}}=\sum_{m=0}^{n}\begin{bmatrix} n \\ m \end{bmatrix}x^m$$

其中 $x^{\overline{n}}=x(x+1)\cdots(x+n-1)$ 为上升幂。

3. 斯特林数与组合数

$$\begin{Bmatrix} n \\ m \end{Bmatrix}=\frac{1}{m!}\sum_{i=0}^{m}(-1)^{m-i}\binom{m}{i}i^n$$

六、贝尔数

B_n：把 n 个不同元素分成若干非空子集的方案数。

$$B_n = \sum_{m=0}^{n} \left\{ {n \atop m} \right\}$$

递推：

$$B_{n+1} = \sum_{i=0}^{n} \binom{n}{i} B_i$$

指数生成函数：

$$\sum_{n=0}^{\infty} B_n \frac{x^n}{n!} = e^{e^x - 1}$$

前几项：$B_0 = 1, B_1 = 1, B_2 = 2, B_3 = 5, B_4 = 15, B_5 = 52$。

长度为 n 的字符串本质不同的结构数（`xyx`，`ama`，`qwq` 被认为结构是本质相同的）就是 B_n。

七、整数拆分

1. 拆分数 $p(n)$

把 n 拆成若干正整数之和的方案数（顺序不计）。

生成函数：

$$\sum_{n=0}^{\infty} p(n) x^n = \prod_{i=1}^{\infty} \frac{1}{1 - x^i}$$

前几项：$p(0) = 1, p(1) = 1, p(2) = 2, p(3) = 3, p(4) = 5, p(5) = 7$。

2. 拆分为恰好 k 个正整数

等于把 n 拆成最大部分为 k 的方案数。

3. 五边形数定理

$$\prod_{i=1}^{\infty} (1 - x^i) = \sum_{k=-\infty}^{\infty} (-1)^k x^{k(3k-1)/2}$$

可用于 $O(n\sqrt{n})$ 计算拆分数。

八、生成函数

1. 普通生成函数（OGF）

数列 $\{a_n\}$ 的 OGF：

$$A(x) = \sum_{n=0}^{\infty} a_n x^n$$

常见：

$$\sum_{n=0}^{\infty} x^n = \frac{1}{1-x}$$

$$\sum_{n=0}^{\infty} \binom{n+k}{k} x^n = \frac{1}{(1-x)^{k+1}}$$

$$\sum_{n=0}^{\infty} c^n x^n = \frac{1}{1-cx}$$

2. 指数生成函数（EGF）

数列 $\{a_n\}$ 的 EGF：

$$A(x) = \sum_{n=0}^{\infty} a_n \frac{x^n}{n!}$$

常见：

$$\sum_{n=0}^{\infty} \frac{x^n}{n!} = e^x$$

$$\sum_{n=0}^{\infty} n\frac{x^n}{n!} = xe^x$$

$$\sum_{n=0}^{\infty} \binom{n}{m} \frac{x^n}{n!} = \frac{x^m}{m!} e^x$$

3. 卷积

$$c_n = \sum_{i=0}^{n} a_i b_{n-i} \iff C(x) = A(x)B(x)$$

4. 常用变换

- 前缀和：$b_n = \sum_{i=0}^{n} a_i \iff B(x) = \frac{A(x)}{1-x}$。
- 差分：$b_n = a_n - a_{n-1} \iff B(x) = (1-x)A(x)$。

九、鸽巢原理

1. 基本形式

把 $n+1$ 个物体放入 n 个盒子，至少有一个盒子包含 ≥ 2 个物体。

2. 推广

把 n 个物体放入 k 个盒子，至少有一个盒子包含 $\geq \lceil n/k \rceil$ 个物体。

3. 应用

- 任意 $n+1$ 个整数中，必有两个数之差被 n 整除。
- 任意 $n+1$ 个不超过 $2n$ 的正整数中，必有两个数互质。
- 任意 $n+1$ 个不超过 $2n$ 的正整数中，必有一个数整除另一个数。

十、卢卡斯定理与扩展卢卡斯

1. 卢卡斯定理

p 为质数：

$$\binom{n}{m} \equiv \binom{\lfloor n/p \rfloor}{\lfloor m/p \rfloor} \binom{n \bmod p}{m \bmod p} \pmod{p}$$

递归形式：

$$\binom{n}{m} \equiv \prod_i \binom{n_i}{m_i} \pmod{p}$$

其中 n_i, m_i 是 n, m 的 p 进制表示。

2. 扩展卢卡斯

求 $\binom{n}{m} \bmod p$，其中 p 不一定是质数。将 p 分解为质数幂 $p = \prod p_i^{e_i}$，分别求 $\binom{n}{m} \bmod p_i^{e_i}$，再用 CRT 合并。

十一、其他常用结论

1. 错排数

$$D_n = n! \sum_{i=0}^{n} \frac{(-1)^i}{i!}$$

$$D_n = (n-1)(D_{n-1} + D_{n-2}), \quad D_0 = 1, D_1 = 0$$

前几项：$D_0 = 1, D_1 = 0, D_2 = 1, D_3 = 2, D_4 = 9, D_5 = 44$。

2. 圆桌问题

n 个人围圆桌，若旋转视为相同：

$$(n-1)!$$

若旋转和翻转都视为相同：

$$\frac{(n-1)!}{2}$$

3. 正整数解个数

$$x_1 + x_2 + \cdots + x_k = n, \quad x_i \geq 1$$

的解数为：

$$\binom{n-1}{k-1}$$

非负整数解：

$$\binom{n+k-1}{k-1}$$

4. 上取整与下取整

$$\left\lceil \frac{a}{b} \right\rceil = \left\lfloor \frac{a+b-1}{b} \right\rfloor$$

$$\left\lfloor \frac{a}{b} \right\rfloor = \left\lceil \frac{a-b+1}{b} \right\rceil$$

5. 二项式系数奇偶性

$\binom{n}{m}$ 为奇数当且仅当 m 的二进制位是 n 的二进制位的子集，即

$$m \& n = m$$

7. 卡特兰数扩展

从 $(0,0)$ 到 (n,m) 且不越过 $y=x$ 的路径数（$n \geq m$）：

$$\binom{n+m}{m} - \binom{n+m}{m-1}$$

8. 施罗德数

从 $(0,0)$ 到 (n,n) 且不越过对角线，允许水平、垂直、对角线的路径数。前几项：$1, 2, 6, 22, 90, \ldots$。

线性代数

一、矩阵基础

1. 矩阵定义与运算

- 矩阵 $A = (a_{ij})_{m \times n}$ 表示 m 行 n 列。
- 加法：对应位置相加，要求同型。
- 数乘：$kA = (ka_{ij})$。
- 转置：A^T 的第 i 行第 j 列是 a_{ji}。

2. 矩阵乘法

设 A 为 $m \times p$，B 为 $p \times n$，则 $C = AB$ 为 $m \times n$，其中

$$c_{ij} = \sum_{k=1}^{p} a_{ik} b_{kj}$$

性质：

- 结合律：$(AB)C = A(BC)$。
- 分配律：$A(B+C) = AB + AC$。
- 不满足交换律：一般 $AB \neq BA$。
- $(AB)^T = B^T A^T$。
- 下三角矩阵 $\times$ 下三角矩阵还是下三角，可以卡常。

3. 矩阵快速幂

用于加速线性递推。若矩阵大小为 $k \times k$，则计算 A^n 的时间复杂度为 $O(k^3 \log n)$。

```cpp
struct Matrix {
    int n, m;
    vector<vector<ll>> a;
    Matrix operator*(const Matrix& other) const {
        Matrix res{n, other.m, vector<vector<ll>>(n, vector<ll>(other.m, 0))};
        for (int i = 0; i < n; i++)
            for (int k = 0; k < m; k++) if (a[i][k])
                for (int j = 0; j < other.m; j++)
                    res.a[i][j] = (res.a[i][j] + a[i][k] * other.a[k][j]) % MOD;
        return res;
    }
};
```

4. 单位矩阵与逆矩阵

- 单位矩阵 I_n：主对角线为 1，其余为 0。
- 逆矩阵：若存在 B 使得 $AB = BA = I$，则 $B = A^{-1}$。
- 可逆条件：A 为方阵且 $\det(A) \neq 0$。

5. 矩阵转置与对称矩阵

- 对称矩阵：$A^T = A$。
- 反对称矩阵：$A^T = -A$。
- 正交矩阵：$A^T A = AA^T = I$。

二、行列式

1. 定义与性质

- n 阶行列式 $\det(A)$ 或 $|A|$ 是一个标量。
- 性质：
  - 交换两行，行列式变号。
  - 某行乘以 k，行列式乘以 k。
  - 某行加上另一行的 k 倍，行列式不变。
  - 若有两行相同，行列式为 0。
  - $\det(AB) = \det(A)\det(B)$。
  - $\det(A^T) = \det(A)$。
  - $\det(A^{-1}) = 1/\det(A)$。

2. 展开与计算

- 按行展开：$\det(A) = \sum_{j=1}^{n} (-1)^{i+j} a_{ij} M_{ij}$，其中 M_{ij} 是余子式。
- 高斯消元求行列式：将矩阵化为上三角，行列式等于对角线乘积乘以交换次数带来的符号。

3. 常见行列式

- 上三角/下三角：对角线乘积。
- 范德蒙德行列式：

$$V(x_1,\ldots,x_n)=\begin{vmatrix}1 & x_1 & x_1^2 & \cdots & x_1^{n-1}\\ 1 & x_2 & x_2^2 & \cdots & x_2^{n-1}\\ \vdots & \vdots & \vdots & \ddots & \vdots\\ 1 & x_n & x_n^2 & \cdots & x_n^{n-1}\end{vmatrix}=\prod_{1\le i<j\le n}(x_j-x_i)$$

4. 矩阵树定理

- 无向图生成树个数：设 $L=D-A$ 为拉普拉斯矩阵，D 为度数矩阵，A 为邻接矩阵。
- 去掉 L 的任意一行一列，剩余矩阵的行列式即为生成树个数。
- 有向图有类似结论（内向树、外向树）。

三、高斯消元

1. 线性方程组

$$\begin{cases}a_{11}x_1+a_{12}x_2+\cdots+a_{1n}x_n=b_1\\ a_{21}x_1+a_{22}x_2+\cdots+a_{2n}x_n=b_2\\ \vdots\\ a_{m1}x_1+a_{m2}x_2+\cdots+a_{mn}x_n=b_m\end{cases}$$

写成增广矩阵 $(A|b)$。

2. 高斯消元法

- 逐列选择主元（绝对值最大或非零），消去下方元素，化为行阶梯形。
- 回代求解。

3. 高斯-约旦消元

- 将矩阵化为行最简形，直接得到解。

4. 解的情况

- 无解：出现 $0=\text{非零}$ 的行。
- 唯一解：秩 $r=n$。
- 无穷多解：秩 $r<n$，有 $n-r$ 个自由变量。

5. 异或方程组

- 模 2 下的高斯消元，可用 bitset 优化，时间复杂度 $O(n^3/64)$。
- 常用于求解异或方程、线性基。

四、矩阵求逆

1. 逆矩阵存在条件

- A 为方阵且 $\det(A)\neq 0$（满秩）。

2. 高斯-约旦求逆

- 构造增广矩阵 $(A|I)$，对 A 进行行变换化为 I，右侧即为 A^{-1}。
- 若左侧无法化为 I，则不可逆。

3. 伴随矩阵

- $A^{-1} = \frac{1}{\det(A)}\text{adj}(A)$，其中 $\text{adj}(A)$ 为伴随矩阵，元素为代数余子式。
- 计算复杂度高，一般用高斯消元。

4. 克莱姆法则

- 若 A 可逆，则 $x_i = \frac{\det(A_i)}{\det(A)}$，其中 A_i 是将 A 的第 i 列替换为 b。
- 理论意义大，实际计算复杂度高。

五、线性基（异或空间）

1. 定义

- 线性基是一组基向量，张成所有可由原集合异或得到的数。
- 通常用二进制表示，每个基向量最高位不同。

2. 插入与查询

```
void insert(ll x) {
    for (int i = 62; i >= 0; i--) {
        if (!(x >> i & 1)) continue;
        if (!basis[i]) { basis[i] = x; return; }
        x ^= basis[i];
    }
}
```

3. 最大异或和

- 从高位到低位贪心：若当前异或和与基向量异或后变大，则异或。

```
ll query_max() {
    ll res = 0;
    for (int i = 62; i >= 0; i--)
        if ((res ^ basis[i]) > res) res ^= basis[i];
    return res;
}
```

4. 第 k 大异或和

- 先将线性基化为最简形式（每个基向量的最高位在其他基向量中为 0）。
- 将基向量按最高位排序，若 k 的二进制第 i 位为 1，则异或对应基向量。

5. 线性基合并

- 将一个线性基的所有基向量插入另一个线性基。

六、特征值与特征向量

1. 定义

- 若存在非零向量 v 使得 $Av = \lambda v$，则 λ 为特征值，v 为特征向量。
- 特征多项式：$\det(\lambda I - A) = 0$。

2. Cayley-Hamilton 定理

- 矩阵满足自己的特征多项式：$p(A) = 0$。

- 可用于加速常系数齐次线性递推。

3. 常系数齐次线性递推

- 递推式 $a_n = \sum_{i=1}^{k} c_i a_{n-i}$。
- 构造转移矩阵，用矩阵快速幂 $O(k^3 \log n)$。
- 或用 BM 算法求最短递推式，再用线性递推优化至 $O(k^2 \log n)$ 或 $O(k \log k \log n)$。

七、矩阵与递推

1. 斐波那契数列

$$\begin{pmatrix} F_{n+1} \\ F_n \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n \begin{pmatrix} F_1 \\ F_0 \end{pmatrix}$$

2. 矩阵快速幂优化 DP

- 若 DP 转移是线性的且每步相同，可写成矩阵形式。
- 例如：$dp_i = \sum_j c_j dp_{i-j}$。

3. 转移矩阵构造

- 状态向量包含当前及之前若干项。
- 转移矩阵根据递推关系构造。

八、特殊矩阵

1. 范德蒙德矩阵

$$V = \begin{pmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^{n-1} \\ 1 & x_2 & x_2^2 & \cdots & x_2^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^{n-1} \end{pmatrix}$$

行列式 $\prod_{i<j}(x_j - x_i)$。

2. 循环矩阵

- 每行是上一行循环右移。
- 可对角化，用 FFT 加速乘法。

3. 托普利茨矩阵

- 每条对角线上的元素相同。

九、线性代数在竞赛中的其他应用

1. 矩阵树定理

- 无向图生成树计数：拉普拉斯矩阵去掉一行一列的行列式。
- 有向图：内向树、外向树。

2. 矩阵乘法与 bitset 优化

- 当矩阵元素为 $0/1$ 且需要模 2 时，可用 bitset 加速，复杂度 $O(n^3/64)$。
- 常用于传递闭包、异或方程组。

3. 线性空间与秩

- 矩阵的秩 = 行秩 = 列秩 = 非零行数（行阶梯形）。
- 秩可用于判断线性相关性、解空间维度。

十、Berlekamp-Massey 算法

1. 用途

- 给定数列前若干项，求最短线性递推式。
- 常用于打表找规律、优化 DP。

2. 算法流程

- 维护当前递推式 C 和上一个递推式 B，以及偏差 d。
- 若当前项符合递推，继续；否则更新递推式。

3. 线性递推优化

- 求出递推式后，可用 Kitamasa 法或多项式取模在 $O(k^2 \log n)$ 或 $O(k \log k \log n)$ 内求第 n 项。

积分

积分结论

$$
\begin{aligned}
B(a,b) &= \int_0^1 x^{a-1}(1-x)^{b-1}\mathrm{d}x && (a,b>0)\\
&= \frac{\Gamma(a)\Gamma(b)}{\Gamma(a+b)}\\
&= \frac{(a-1)!(b-1)!}{(a+b-1)!} && (a,b\in\mathbb{N}^+)
\end{aligned}
$$

数论函数

一、积性函数与完全积性函数

定义

- 若 $\gcd(a,b)=1$ 时，$f(ab)=f(a)f(b)$，则 f 为**积性函数**。
- 若对任意 a,b 都有 $f(ab)=f(a)f(b)$，则 f 为**完全积性函数**。

常见积性函数

- 欧拉函数 $\varphi(n)$
- 莫比乌斯函数 $\mu(n)$
- 约数个数 $d(n)$
- 约数和 $\sigma(n)$
- 单位函数 $\varepsilon(n)=[n=1]$
- 恒等函数 $id(n)=n$
- 常函数 $1(n)=1$

性质

积性函数由其在素数幂 p^k 上的取值完全确定。

二、欧拉函数 $\varphi(n)$

定义

$1\sim n$ 中与 n 互质的数的个数。

公式

$$\varphi(n)=n\prod_{p|n}\left(1-\frac{1}{p}\right)$$

性质

$$\varphi(p^k)=p^k-p^{k-1}$$

$$\sum_{d|n}\varphi(d)=n$$

$$\varphi(n)=\sum_{d|n}\mu(d)\cdot\frac{n}{d}$$

线性筛求 φ

```
const int N = 1e6 + 5;
int phi[N], primes[N], cnt;
bool vis[N];

void init_phi(int n) {
    phi[1] = 1;
    for (int i = 2; i <= n; i++) {
        if (!vis[i]) {
            primes[++cnt] = i;
            phi[i] = i - 1;
        }
        for (int j = 1; j <= cnt && i * primes[j] <= n; j++) {
            vis[i * primes[j]] = true;
            if (i % primes[j] == 0) {
                phi[i * primes[j]] = phi[i] * primes[j];
                break;
            } else {
                phi[i * primes[j]] = phi[i] * (primes[j] - 1);
            }
        }
```

```
    }
}
```

三、莫比乌斯函数 $\mu(n)$

定义

$$\mu(n)=\begin{cases}1, & n=1\\ 0, & n\text{ 含有平方因子}\\ (-1)^k, & n\text{ 是 }k\text{ 个不同素数的乘积}\end{cases}$$

性质

$$\sum_{d|n}\mu(d)=[n=1]$$

$$\mu*1=\varepsilon$$

线性筛求 μ

```
int mu[N], primes[N], cnt;
bool vis[N];

void init_mu(int n) {
    mu[1] = 1;
    for (int i = 2; i <= n; i++) {
        if (!vis[i]) {
            primes[++cnt] = i;
            mu[i] = -1;
        }
        for (int j = 1; j <= cnt && i * primes[j] <= n; j++) {
            vis[i * primes[j]] = true;
            if (i % primes[j] == 0) {
                mu[i * primes[j]] = 0;
                break;
            } else {
                mu[i * primes[j]] = -mu[i];
            }
        }
    }
}
```

四、约数个数 $d(n)$ 与约数和 $\sigma(n)$

约数个数

若 $n=\prod p_i^{e_i}$，则

$$d(n)=\prod(e_i+1)$$

约数和

$$\sigma(n)=\prod\frac{p_i^{e_i+1}-1}{p_i-1}$$

线性筛求 $d(n)$

```
int d[N], num[N], primes[N], cnt;
bool vis[N];

void init_d(int n) {
    d[1] = 1;
    for (int i = 2; i <= n; i++) {
        if (!vis[i]) {
            primes[++cnt] = i;
            d[i] = 2;
            num[i] = 1; // 最小素因子的指数
        }
        for (int j = 1; j <= cnt && i * primes[j] <= n; j++) {
            vis[i * primes[j]] = true;
            if (i % primes[j] == 0) {
                num[i * primes[j]] = num[i] + 1;
                d[i * primes[j]] = d[i] / (num[i] + 1) * (num[i] + 2);
                break;
            } else {
                num[i * primes[j]] = 1;
                d[i * primes[j]] = d[i] * 2;
            }
        }
    }
}
```

线性筛求 $\sigma(n)$

```
ll sigma[N], sum_p[N];
int primes[N], cnt;
bool vis[N];

void init_sigma(int n) {
    sigma[1] = 1;
    for (int i = 2; i <= n; i++) {
        if (!vis[i]) {
            primes[++cnt] = i;
            sigma[i] = i + 1;
            sum_p[i] = i + 1;
        }
        for (int j = 1; j <= cnt && i * primes[j] <= n; j++) {
            vis[i * primes[j]] = true;
            if (i % primes[j] == 0) {
                sum_p[i * primes[j]] = sum_p[i] * primes[j] + 1;
                sigma[i * primes[j]] = sigma[i] / sum_p[i] * sum_p[i * primes[j]];
                break;
            } else {
                sum_p[i * primes[j]] = primes[j] + 1;
                sigma[i * primes[j]] = sigma[i] * (primes[j] + 1);
            }
        }
    }
}
```

五、狄利克雷卷积

定义

$$(f*g)(n)=\sum_{d|n}f(d)g\left(\frac{n}{d}\right)$$

性质

- 交换律：$f*g=g*f$
- 结合律：$(f*g)*h=f*(g*h)$
- 单位元：$\varepsilon*f=f$
- 若 f,g 积性，则 $f*g$ 也积性。

常见卷积

$$\mu*1=\varepsilon$$

$$\varphi*1=id$$

$$d=1*1$$

$$\sigma=id*1$$

六、莫比乌斯反演

形式一

若

$$F(n)=\sum_{d|n}f(d)$$

则

$$f(n)=\sum_{d|n}\mu(d)F\left(\frac{n}{d}\right)$$

形式二

若

$$F(n)=\sum_{n|d}f(d)$$

则

$$f(n)=\sum_{n|d}\mu\left(\frac{d}{n}\right)F(d)$$

常见应用

$$\sum_{i=1}^{n}\sum_{j=1}^{m}[\gcd(i,j)=1]=\sum_{d=1}^{\min(n,m)}\mu(d)\left\lfloor\frac{n}{d}\right\rfloor\left\lfloor\frac{m}{d}\right\rfloor$$

七、杜教筛

用于在 $O(n^{2/3})$ 时间内求积性函数前缀和 $S(n)=\sum_{i=1}^{n} f(i)$。

公式

若 $h=f*g$，则

$$g(1)S(n)=\sum_{i=1}^{n} h(i)-\sum_{i=2}^{n} g(i)S\left(\left\lfloor \frac{n}{i} \right\rfloor\right)$$

常用

- 求 μ 前缀和：取 $g=1$，$h=\varepsilon$，则

$$S(n)=1-\sum_{i=2}^{n} S\left(\left\lfloor \frac{n}{i} \right\rfloor\right)$$

- 求 φ 前缀和：取 $g=1$，$h=id$，则

$$S(n)=\frac{n(n+1)}{2}-\sum_{i=2}^{n} S\left(\left\lfloor \frac{n}{i} \right\rfloor\right)$$

模板

```
const int N = 5e6 + 5;
ll phi[N], mu[N];
int primes[N], cnt;
bool vis[N];
unordered_map<ll, ll> mp_phi, mp_mu;

void init(int n) {
    phi[1] = mu[1] = 1;
    for (int i = 2; i <= n; i++) {
        if (!vis[i]) {
            primes[++cnt] = i;
            phi[i] = i - 1;
            mu[i] = -1;
        }
        for (int j = 1; j <= cnt && i * primes[j] <= n; j++) {
            vis[i * primes[j]] = true;
            if (i % primes[j] == 0) {
                phi[i * primes[j]] = phi[i] * primes[j];
                mu[i * primes[j]] = 0;
                break;
            } else {
                phi[i * primes[j]] = phi[i] * (primes[j] - 1);
                mu[i * primes[j]] = -mu[i];
            }
        }
    }
    for (int i = 1; i <= n; i++) {
        phi[i] += phi[i - 1];
        mu[i] += mu[i - 1];
    }
}

ll get_phi(ll n) {
```

```
    if (n < N) return phi[n];
    if (mp_phi.count(n)) return mp_phi[n];
    ll ans = n * (n + 1) / 2;//可能爆ll
    for (ll l = 2, r; l <= n; l = r + 1) {
        r = n / (n / l);
        ans -= (r - l + 1) * get_phi(n / l);
    }
    return mp_phi[n] = ans;
}

ll get_mu(ll n) {
    if (n < N) return mu[n];
    if (mp_mu.count(n)) return mp_mu[n];
    ll ans = 1;
    for (ll l = 2, r; l <= n; l = r + 1) {
        r = n / (n / l);
        ans -= (r - l + 1) * get_mu(n / l);
    }
    return mp_mu[n] = ans;
}
```

八、min_25 筛

用于求积性函数前缀和，要求 $f(p)$ 是关于 p 的多项式，且 $f(p^k)$ 容易计算。复杂度 $O(n^{3/4}/\log n)$。

核心思想

1. 预处理素数，计算 $g(n,j)$：筛掉前 j 个素数后，剩下数的 f 值之和（f 取多项式形式）。
2. 计算 $S(n,j)$：最小素因子 $\geq p_j$ 的数的 f 值之和。

模板框架（以求 φ 前缀和为例）

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

const int N = 1e6 + 5;
const ll MOD = 1e9 + 7;          // 按题目要求修改
const ll inv2 = (MOD + 1) / 2;   // MOD 为奇质数时可用

ll n, sqr;
ll w[N], g1[N], g2[N], id1[N], id2[N];
int primes[N], cnt;
bool vis[N];
ll sp1[N], sp2[N];  // 素数前缀和：sp1 = sum p, sp2 = sum 1

int get_id(ll x) {
    return x <= sqr ? id1[x] : id2[n / x];
}

// 线性筛预处理素数及素数前缀和
void init() {
    for (int i = 2; i <= sqr; i++) {
        if (!vis[i]) primes[++cnt] = i;
        for (int j = 1; j <= cnt && 1LL * i * primes[j] <= sqr; j++) {
            vis[i * primes[j]] = true;
```

```
            if (i % primes[j] == 0) break;
        }
    }
    for (int i = 1; i <= cnt; i++) {
        sp1[i] = (sp1[i - 1] + primes[i]) % MOD;
        sp2[i] = (sp2[i - 1] + 1) % MOD;
    }
}

// S(x, y): 最小素因子 >= primes[y] 的数的 f 值之和
ll S(ll x, int y) {
    if (primes[y] >= x) return 0;
    int k = get_id(x);
    // 初始: g1 - g2 表示所有素数的 φ 之和，减去前 y 个素数的贡献
    ll ans = (g1[k] - g2[k] - sp1[y] + sp2[y]) % MOD;
    for (int i = y + 1; i <= cnt && 1LL * primes[i] * primes[i] <= x; i++) {
        ll p = primes[i];
        ll pe = p;  // pe = p^e
        for (int e = 1; pe * p <= x; e++, pe *= p) {
            // φ(p^e) = pe - pe / p
            ll phi_pe = (pe - pe / p) % MOD;
            // φ(p^{e+1}) = pe * p - pe
            ll phi_pe1 = (pe * p - pe) % MOD;
            ans = (ans + S(x / pe, i) * phi_pe + phi_pe1) % MOD;
        }
    }
    return (ans % MOD + MOD) % MOD;
}

ll min25() {
    sqr = sqrt(n);
    init();

    int tot = 0;
    for (ll l = 1, r; l <= n; l = r + 1) {
        r = n / (n / l);
        w[++tot] = n / l;
        if (w[tot] <= sqr) id1[w[tot]] = tot;
        else id2[n / w[tot]] = tot;

        // 初始化 g1, g2: 假设所有数都是素数
        // g1 = sum_{i=2}^{w} i = w(w+1)/2 - 1
        // g2 = sum_{i=2}^{w} 1 = w - 1
        ll wm = w[tot] % MOD;
        g1[tot] = (wm * ((w[tot] + 1) % MOD) % MOD * inv2 % MOD - 1 + MOD) % MOD;
        g2[tot] = (wm - 1 + MOD) % MOD;
    }

    // 筛 g
    for (int j = 1; j <= cnt; j++) {
        ll p = primes[j];
        for (int i = 1; i <= tot && p * p <= w[i]; i++) {
            int k = get_id(w[i] / p);
            g1[i] = (g1[i] - p % MOD * ((g1[k] - sp1[j - 1]) % MOD + MOD) % MOD) %
MOD;
            g2[i] = (g2[i] - ((g2[k] - sp2[j - 1]) % MOD + MOD) % MOD) % MOD;
        }
```

```
    }

    // 最终答案：ϕ(1) = 1
    ll ans = (S(n, 0) + 1) % MOD;
    return (ans + MOD) % MOD;
}

int main() {
    cin >> n;
    cout << min25() << endl;
    return 0;
}
```

注意：具体实现需根据 $f(p)$ 和 $f(p^k)$ 调整。

九、洲阁筛

洲阁筛是另一种求积性函数前缀和的筛法，复杂度 $O(n^{3/4}/\log n)$，适用于 $f(p)$ 为多项式或可快速计算的情况。现在通常被 min_25 筛替代，但某些题目可能用到。

基本思想

将数分为 $\leq \sqrt{n}$ 和 $> \sqrt{n}$ 两部分，分别处理。

现状

实现复杂，使用频率低，建议优先掌握杜教筛和 min_25 筛。

十、常见积性函数

定义回顾

- **积性函数**：若 $\gcd(a,b)=1$，则 $f(ab)=f(a)f(b)$。
- **完全积性函数**：对任意 a,b，都有 $f(ab)=f(a)f(b)$。
- 积性函数由其在素数幂 p^k 上的取值完全确定。

符号约定：

- 函数参数用 k 或 s；
- 素数幂统一写为 p^e，其中 $e \geq 1$；
- $*$ 表示狄利克雷卷积。

名称	符号	定义	素数幂 p^e 上的值（$e \geq 1$）	积性	完全积性
欧拉函数	$\varphi(n)$	$1 \sim n$ 中与 n 互质的数的个数	$\varphi(p^e)=p^e-p^{e-1}$	是	否

名称	符号	定义	素数幂 p^e 上的值（$e \geq 1$）	积性	完全积性
莫比乌斯函数	$\mu(n)$	$n=1$ 时为 1；含平方因子为 0；k 个不同素因子为 $(-1)^k$	$\mu(p)=-1,\ \mu(p^e)=0\ (e\geq 2)$	是	否
约数个数	$d(n)$ 或 $\tau(n)$	$\sum_{d\mid n} 1$	$d(p^e)=e+1$	是	否
约数和	$\sigma(n)$	$\sum_{d\mid n} d$	$\sigma(p^e)=\dfrac{p^{e+1}-1}{p-1}$	是	否
广义约数和	$\sigma_k(n)$	$\sum_{d\mid n} d^k$	$\sigma_k(p^e)=\dfrac{p^{k(e+1)}-1}{p^k-1}$	是	否
单位函数	$\varepsilon(n)$	$[n=1]$	$\varepsilon(1)=1,\ \varepsilon(p^e)=0\ (e\geq 1)$	是	是
恒等函数	$id(n)$	n	$id(p^e)=p^e$	是	是
常函数	$1(n)$	1	$1(p^e)=1$	是	是
幂函数	$id_k(n)$	n^k	$id_k(p^e)=p^{ke}$	是	是
刘维尔函数	$\lambda(n)$	$(-1)^{\Omega(n)}$，$\Omega(n)$ 为总素因子个数（带重数）	$\lambda(p^e)=(-1)^e$	是	是
无平方因子指示	$\lvert\mu(n)\rvert$	n 无平方因子时为 1，否则为 0	$\lvert\mu(p)\rvert=1,\ \lvert\mu(p^e)\rvert=0\ (e\geq 2)$	是	否
$2^{\omega(n)}$	$2^{\omega(n)}$	$\omega(n)$ 为不同素因子个数	$2^{\omega(p^e)}=2$	是	否

补充说明

- **完全积性函数**只有：$\varepsilon,\ 1,\ id,\ id_k,\ \lambda$。
- $\varphi,\ \mu,\ d,\ \sigma,\ \sigma_k,\ \lvert\mu\rvert,\ 2^{\omega}$ 都是积性但非完全积性。
- 常见卷积关系：

$$\varepsilon=\mu*1,\quad id=\varphi*1,\quad d=1*1,\quad \sigma=id*1,$$

$$\sigma_k=id_k*1,\quad \varphi=\mu*id,\quad \lvert\mu\rvert*1=2^{\omega(n)}.$$

- 加性函数（不是积性函数）：$\omega(n),\ \Omega(n),\ \nu_p(n)$。但由它们构造的 $2^{\omega(n)}$、$(-1)^{\Omega(n)}$ 可以是积性函数。

完全积性函数补充

完全积性函数要求对任意 a,b 都满足 $f(ab)=f(a)f(b)$。常见的有：

- $\varepsilon(n)=[n=1]$

- $1(n)=1$
- $id(n)=n$
- $id_k(n)=n^k$
- $\lambda(n)=(-1)^{\Omega(n)}$

注意：φ,μ,d,σ 都不是完全积性函数。

常见卷积关系

这些关系在莫比乌斯反演、杜教筛中非常常用：

$$\varepsilon=\mu*1$$

$$id=\varphi*1$$

$$d=1*1$$

$$\sigma=id*1$$

$$\sigma_k=id_k*1$$

$$\varphi=\mu*id$$

$$|\mu|*1=2^{\omega(n)}$$

其中 $*$ 表示狄利克雷卷积：

$$(f*g)(n)=\sum_{d|n}f(d)g\left(\frac{n}{d}\right)$$

容易混淆的加性函数

以下函数是**加性函数**，不是积性函数：

- $\omega(n)$：不同素因子个数，满足 $\omega(ab)=\omega(a)+\omega(b)$（当 $\gcd(a,b)=1$）。
- $\Omega(n)$：总素因子个数（带重数），满足 $\Omega(ab)=\Omega(a)+\Omega(b)$（对任意 a,b）。
- $\nu_p(n)$：n 中素因子 p 的指数。

但由它们构造的一些函数可以是积性，例如 $2^{\omega(n)}$、$(-1)^{\Omega(n)}$。

线性筛中常见需要维护的辅助数组

对于积性函数线性筛，通常需要维护：

- 最小素因子 p；
- 最小素因子的指数 e；
- 最小素因子幂 p^e；
- 去掉最小素因子后的部分 n/p^e。

例如求 $\varphi(n)$ 时：

- 若 $i\%p=0$，则 $\varphi(ip)=\varphi(i)\cdot p$；
- 否则 $\varphi(ip)=\varphi(i)\cdot(p-1)$。

求 $d(n)$ 时：

- 维护最小素因子指数 $num[i]$；
- 若 $i\%p=0$，$d(ip)=d(i)/(num[i]+1)\cdot(num[i]+2)$；
- 否则 $d(ip)=d(i)\cdot 2$。

数论

常见数字

x 范围	与 x 互质个数	x 的因数个数	x 的因数和	x 质因子个数	x 质因子幂次最高项
10^5	$\sim 10^5$	128	4×10^5	6	16
10^9	$\sim 10^9$	1344	4×10^9	9	29
10^{18}	$\sim 10^{18}$	10^5	6×10^{18}	15	59

二次剩余

适用条件：

- p 是奇素数。
- 求满足

$$x^2\equiv n \pmod{p}$$

的 x。

- `cipolla(n, p)` 有解返回一个根，无解返回 `-1`。
- 若返回一个根 x，另一个根为

$$(p-x)\bmod p$$

- 若 $x=0$，两个根重合。
- 模合数不能直接使用本模板。

用法示例：

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long p = 1000000007;
    long long n = 12345;

    long long x = cipolla(n, p);
    if (x == -1) {
        cout << "No solution\n";
    } else {
        long long y = (p - x) % p;
        cout << x << ' ' << y << '\n';
        // 验证：x^2 == n mod p
```

```
    }
    return 0;
}
```

原理简述

欧拉判别法：对奇素数 (p)，

$$n^{\frac{p-1}{2}} \equiv \begin{cases} 1 \pmod p, & n \text{ 是二次剩余}, \\ -1 \pmod p, & n \text{ 不是二次剩余}, \\ 0 \pmod p, & n \equiv 0 \pmod p. \end{cases}$$

Cipolla 算法：找一个 (a)，使得

$$w = a^2 - n$$

是模 (p) 的非二次剩余。然后在扩域 (\mathbb F_p[\sqrt w]) 中计算

$$(a + \sqrt{w})^{\frac{p+1}{2}} = x + 0\sqrt{w}$$

则实部 (x) 就是

$$x^2 \equiv n \pmod p$$

的一个解。

扩域乘法：

$$(x_1 + y_1\sqrt{w})(x_2 + y_2\sqrt{w}) = (x_1x_2 + y_1y_2w) + (x_1y_2 + x_2y_1)\sqrt{w}$$

代码模板

```cpp
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
using i128 = __int128_t;

ll fp(ll a, ll b, ll mod) {
    ll res = 1;
    a %= mod;
    if (a < 0) a += mod;
    while (b) {
        if (b & 1) res = (i128)res * a % mod;
        a = (i128)a * a % mod;
        b >>= 1;
    }
    return res;
}

// 返回 1: n 是二次剩余
// 返回 -1: n 不是二次剩余
// 返回 0: n == 0
ll lcheck(ll n, ll p) {
    n %= p;
    if (n < 0) n += p;
    if (n == 0) return 0;
    ll r = fp(n, (p - 1) / 2, p);
    return (r == 1 ? 1 : -1);
```

```
}

struct Complex {
    ll x, y; // x + y * sqrt(W)
};

ll W;

Complex ext_mul(Complex a, Complex b, ll p) {
    Complex c;
    c.x = ((i128)a.x * b.x + ((i128)a.y * b.y % p) * W) % p;
    c.y = ((i128)a.x * b.y + (i128)a.y * b.x) % p;
    return c;
}

Complex ext_pow(Complex a, ll b, ll p) {
    Complex res{ 1, 0 };
    while (b) {
        if (b & 1) res = ext_mul(res, a, p);
        a = ext_mul(a, a, p);
        b >>= 1;
    }
    return res;
}

// 求 x^2 = n (mod p)，p 为奇素数
// 有解返回一个 x，无解返回 -1
ll cipolla(ll n, ll p) {
    n %= p;
    if (n < 0) n += p;

    if (p == 2) return n; // 模 2 时 0/1 都有解
    if (n == 0) return 0;
    if (lcheck(n, p) != 1) return -1;

    static mt19937_64 rng(
        chrono::steady_clock::now().time_since_epoch().count()
    );

    ll a;
    while (true) {
        a = (ll)(rng() % (uint64_t)p);
        W = (ll)(((i128)a * a - n) % p);
        if (W < 0) W += p;
        if (lcheck(W, p) == -1) break;
    }

    Complex t{ a, 1 };
    Complex ans = ext_pow(t, (p + 1) / 2, p);

    // 理论上 ans.y == 0
    return ans.x;
}
```

复杂度：期望 $O(\log p)$

数域扩充

斐波那契数列有通项公式：

$$F_n = \frac{1}{\sqrt{5}}(\alpha^n - \beta^n)$$

$$\begin{cases} \alpha = \dfrac{1+\sqrt{5}}{2} \\ \beta = \dfrac{1-\sqrt{5}}{2} \end{cases}$$

（实际上 α 和 β 是方程 $x^2 - x - 1 = 0$ 的两个根，所以有 $\alpha + \beta = 1$，$\alpha\beta = -1$，$\alpha^2 = \alpha + 1$）

现在尝试使用快速幂求出模 $10^9 + 7$ 意义下的 F_n；

由于在该模数意义下，不存在 $x^2 \equiv 5 \pmod{10^9 + 7}$，但是我们可以通过扩充数域，令 $r^2 \equiv 5 \pmod{10^9 + 7}$，于是所有数可以写作 $a + b \cdot r$，将其作为复数乘法，计算快速幂即可。

对于多个复数 r, s，需要表示为 $a + b \cdot r + c \cdot s + d \cdot rs$。

对于逆元：

- 已知不存在 $x^2 \equiv d \pmod{p}$，并定义 $r^2 \equiv d \pmod{p}$；
- 定义共轭：$\overline{z} = a - br$；
- 定义范数：$N(z) = z\overline{z} = a^2 - db^2$；
- p 是素数且 $N(z) \neq 0$ 时，有 $z^{-1} = \dfrac{\overline{z}}{N(z)} = \dfrac{a - br}{a^2 - db^2}$；
- 分母用费马小定理求出；
- 若保证 p 是素数且不存在 $x^2 \equiv d \pmod{p}$，那么所有非零元素都有逆元；

复数乘法板子

```
template<int MOD, int D>
struct Complex {
    int a, b;// a + bi
    static constexpr int mod = MOD;
    static constexpr int d = D;//i^2 = d

    Complex(ll a_ = 0, ll b_ = 0) {
        a = a_ % mod; if (a < 0) a += mod;
        b = b_ % mod; if (b < 0) b += mod;
    }

    Complex& operator+=(const Complex& o) {
        a += o.a; if (a >= mod) a -= mod;
        b += o.b; if (b >= mod) b -= mod;
        return *this;
    }

    Complex& operator-=(const Complex& o) {
        a -= o.a; if (a < 0) a += mod;
        b -= o.b; if (b < 0) b += mod;
        return *this;
    }

    Complex& operator*=(const Complex& o) {
        int na = (1LL * a * o.a + 1LL * b * o.b % mod * d) % mod;
```

```
        int nb = (1LL * a * o.b + 1LL * b * o.a) % mod;
        a = na;
        b = nb;
        return *this;
    }

    friend Complex operator+(Complex x, const Complex& y) { return x += y; }
    friend Complex operator-(Complex x, const Complex& y) { return x -= y; }
    friend Complex operator*(Complex x, const Complex& y) { return x *= y; }

    friend bool operator==(const Complex& x, const Complex& y) {
        return x.a == y.a && x.b == y.b;
    }
    friend bool operator!=(const Complex& x, const Complex& y) {
        return !(x == y);
    }
};

template<int MOD, int D>
Complex<MOD, D> fpow(Complex<MOD, D> a, ll b) {
    Complex<MOD, D> r(1);
    while (b) {
        if (b & 1) r *= a;
        a *= a;
        b >>= 1;
    }
    return r;
}
```

图论

各种图

竞赛图

竞赛图是指 n 个点，$\frac{n(n-1)}{2}$ 条边的有向图，其中任意一对点都恰好有一条有向边。

性质：

- 缩点之后是一条广义链，一条主链和一些方向和主链一致的边；并且拓扑序小的 SCC 向所有拓扑序比他大的 SCC 都有边；
- 若 u 的出度大于 v 的出度，则 u 能到达 v；
- 竞赛图存在一条哈密顿路径，竞赛图的任意一个 SCC 存在一条哈密顿回路；
- 竞赛图的一个大小至少是 k 的 SCC，存在 3 至 k 元环；
- 令 s_i 为第 i 个点的出度，对 s 排序之后，若满足 $\sum\limits_{i=1}^{k} s_i \geq \binom{k}{2}$ 且 $\sum\limits_{i=1}^{n} s_i = \binom{n}{2}$，则存在竞赛图。
- 三元环个数：$\binom{n}{3} - \sum\limits_{i} \binom{out_i}{2}$；
- n 个点强连通竞赛图的个数 g_n，记 $f_i = 2^{i(i-1)/2}$ 为 i 个点竞赛图个数，有 $g_n = f_n - \sum\limits_{i=1}^{n-1} g_i \cdot f_{n-i} \cdot \binom{n}{i}$；
- 123

线段树优化建图

解决区间向区间连边的问题

建立两棵线段树 U, D，U 儿子向父亲连边，D 父亲向儿子连边。

每个叶子节点，D 向 U 连边。

对于边 $x \to y$，从 U 代表 x 点向 D 代表 y 点连边。区间同理。

```
#define ls x<<1
#define rs x<<1|1
struct segTree {
    struct Node {
        int id, l, r, fa;
    };
    vector<Node> tr, leafId;
    void init(int n, int& cnt) {
        tr.assign(4 * n + 20, Node());
        build(1, 1, n, 0, cnt);
    }
    void build(int x, int l, int r, int fa, int& cnt) {
        tr[x] = { ++cnt, l,r,fa };
        if (l == r) {
            leafId.push_back(tr[x]);
            return;
        }
```

```
        int mid = l + r >> 1;
        build(ls, l, mid, tr[x].id, cnt);
        build(rs, mid + 1, r, tr[x].id, cnt);
    }
};
struct segGTree {
    int treecnt;
    //int sz, n;

    segTree U, D;
    vector<vector<int>> graph;

    segGTree(int n) {
        treecnt = 0;
        U.init(n, treecnt);
        D.init(n, treecnt);
        graph.assign(treecnt + 20, vector<int>());
        for (int i = 0; i < U.leafId.size(); i++) {
            graph[D.leafId[i].id].push_back(U.leafId[i].id);
            //graph[U.leafId[i].id].push_back(D.leafId[i].id); 可选
        }
        for (auto [id, l, r, faid] : D.tr) {
            if (faid == 0 || id == 0) continue;
            graph[faid].push_back(id);
        }
        for (auto [id, l, r, faid] : U.tr) {
            if (faid == 0 || id == 0) continue;
            graph[id].push_back(faid);
        }
    }

    int findId(int x, int pos, segTree& tree) {
        if (tree.tr[x].l == tree.tr[x].r) return tree.tr[x].id;
        int mid = tree.tr[x].l + tree.tr[x].r >> 1;
        if (pos <= mid) return findId(ls, pos, tree);
        else return findId(rs, pos, tree);
    }
    void makeEdge(int x, int from, int ql, int qr, segTree& tree, bool p2s) {
        if (ql <= tree.tr[x].l && tree.tr[x].r <= qr) {
            if (p2s) graph[from].push_back(tree.tr[x].id);
            else graph[tree.tr[x].id].push_back(from);
            return;
        }
        int mid = tree.tr[x].l + tree.tr[x].r >> 1;
        if (ql <= mid) makeEdge(ls, from, ql, qr, tree, p2s);
        if (qr > mid) makeEdge(rs, from, ql, qr, tree, p2s);
    }
    void pos2seg(int pos, int ql, int qr) {
        int ID = findId(1, pos, U);
        makeEdge(1, ID, ql, qr, D, 1);
    }
    void seg2pos(int ql, int qr, int pos) {
        int ID = findId(1, pos, D);
        makeEdge(1, ID, ql, qr, U, 0);
    }
};
```

长链剖分

对于每个节点，选择子树高度（最深深度）最大的儿子作为长儿子，其余儿子作为轻儿子。长儿子形成的链称为轻链。

性质：

- 长链长度和为 n；
- 长链长度 = 链顶子树高度；
- 轻边使得长链长度严格增加；
- **任意点到根的轻边数为** $O(\sqrt{n})$；

作用：

- 树上 k 级祖先：$O(n \log n) \sim O(1)$

  预处理倍增数组，并对每条长链的链顶保存向上、向下各 链长 个信息。查询 u 的 k 级祖先时，先倍增跳到 u 的 2^j 级祖先 v，使得剩余步数小于 2^j。由长链性质，v 所在长链的链长至少为 2^j，因此可以 $O(1)$ 从链顶数组中找到答案。

- 优化深度相关 DP：

  123

数据结构

线段树

维护区间和的懒标记清空

```
struct SegTree {
    struct Node {
        int tag;      // 时间戳
        long long sum;
        long long lazy; // 懒标记
    } tree[4 * MAXN];
    int curTag = 0;

    void clear() { curTag++; }

    void resetNode(int p) {
        if (tree[p].tag != curTag) {
            tree[p].tag = curTag;
            tree[p].sum = 0;
            tree[p].lazy = 0;
            // 其他需要重置的字段
        }
    }

    void pushdown(int p, int l, int r) {
        resetNode(p);//
        if (l == r) return;
        int mid = (l + r) >> 1;
        resetNode(p << 1);//
        resetNode(p << 1 | 1);//
        if (tree[p].lazy != 0) {
            // 下推懒标记，注意左右孩子可能刚被 reset，需要先 reset 再应用
            tree[p << 1].sum += tree[p].lazy * (mid - l + 1);
            tree[p << 1].lazy += tree[p].lazy;
            tree[p << 1 | 1].sum += tree[p].lazy * (r - mid);
            tree[p << 1 | 1].lazy += tree[p].lazy;
            tree[p].lazy = 0;
        }
    }

    void update(int p, int l, int r, int ql, int qr, int val) {
        resetNode(p);//
        if (ql <= l && r <= qr) {
            tree[p].sum += val * (r - l + 1);
            tree[p].lazy += val;
            return;
        }
        pushdown(p, l, r);
        int mid = (l + r) >> 1;
        if (ql <= mid) update(p << 1, l, mid, ql, qr, val);
        if (qr > mid) update(p << 1 | 1, mid + 1, r, ql, qr, val);
        tree[p].sum = tree[p << 1].sum + tree[p << 1 | 1].sum;
    }
```

```
    long long query(int p, int l, int r, int ql, int qr) {
        resetNode(p);//
        if (ql <= l && r <= qr) return tree[p].sum;
        pushdown(p, l, r);
        int mid = (l + r) >> 1;
        long long res = 0;
        if (ql <= mid) res += query(p << 1, l, mid, ql, qr);
        if (qr > mid) res += query(p << 1 | 1, mid + 1, r, ql, qr);
        return res;
    }
    //pushup也要reset左右子节点
};
```

树状数组

前缀最大值

```
const int INF = 1e9;
int n;
vector<int> c; // 树状数组，初始化为 -INF

int lowbit(int x) {
    return x & -x;
}

// 单点更新：把位置 x 的值改为 val
// !!!!!要求 val >= 原值（维护最大值时）!!!!!，否则更新会失败
void update(int x, int val) {
    while (x <= n) {
        c[x] = max(c[x], val);
        x += lowbit(x);
    }
}

// 查询前缀 [1, x] 的最大值
int query(int x) {
    int res = -INF;
    while (x > 0) {
        res = max(res, c[x]);
        x -= lowbit(x);
    }
    return res;
}
```

时间戳优化（多测清空等）

```
// 时间戳优化
int tag[MAXN], t[MAXN], Tag;

void reset() { ++Tag; }

void add(int k, int v) {
  while (k <= n) {
    if (tag[k] != Tag) t[k] = 0;
```

```
    t[k] += v, tag[k] = Tag;
    k += lowbit(k);
  }
}

int getsum(int k) {
  int ret = 0;
  while (k > 0) {
    if (tag[k] == Tag) ret += t[k];
    k -= lowbit(k);
  }
  return ret;
}
```

CDQ分治

三位偏序：f_i 代表满足 $j \neq i, a_j \leq a_i, b_j \leq b_i, c_j \leq c_i$ 的数量，下面的代码会对 f_i 做桶。

思路：将 $[l, r]$ 内的数对分为三类，$i < j < mid$，$mid < i < j$ 和 $i < mid < j$，前两种递归解决，第三种使用归并+数据结构维护。

注意考虑相同数字的贡献

```
int n, k, m;
array<int, 3> arr[maxn];
array<int, 5> brr[maxn], tmp[maxn];

int d[maxn];

struct BIT {...}T; // 单点加区间查

void cdq(int l, int r) {
    if (l == r) {
        auto& [a, b, c, cnt, val] = brr[l];
        val = cnt - 1;
        return;
    }
    int mid = l + r >> 1;
    cdq(l, mid);
    cdq(mid + 1, r);
    // 分治会让两个区间内部分别按 b 有序，于是可以在归并的时候利用数据结构统计 ci <= cj 的数量
    int i = l, j = mid + 1;

    int tmpcnt = l;
    for (; j <= r; j++) {
        while (i <= mid && brr[i][1] <= brr[j][1]) {
            T.add(brr[i][2], brr[i][3]);
            tmp[tmpcnt++] = brr[i];
            i++;
        }
        brr[j][4] += T.sum(1, brr[j][2]);
        tmp[tmpcnt++] = brr[j];
    }
    for (int x = l; x < i; x++) T.add(brr[x][2], -brr[x][3]);
    while (i <= mid) { tmp[tmpcnt++] = brr[i++]; }
```

```
    while (j <= r) { tmp[tmpcnt++] = brr[j++]; }
    for (int i = l; i <= r; i++) {
        brr[i] = tmp[i];
    }

}

void solve() {
    cin >> n >> k;// k 是值域大小
    for (int i = 1; i <= n; i++) {
        cin >> arr[i][0] >> arr[i][1] >> arr[i][2];
    }
    sort(arr + 1, arr + 1 + n);
    // 一定要去重
    for (int i = 1; i <= n; ) {
        int j = i;
        while (j <= n && arr[j] == arr[i]) j++;
        brr[++m] = { arr[i][0], arr[i][1], arr[i][2], j - i, 0 };
        i = j;
    }
    sort(brr + 1, brr + 1 + m);
    // 按照a排序，这样对于i in [l,mid],j in [mid+1,r]有 ai < aj
    T.init(k+1);
    cdq(1, m);
    for (int i = 1; i <= m; i++) {
        auto& [a, b, c, cnt, val] = brr[i];
        d[val] += cnt;
    }
    for (int i = 0; i < n; i++) cout << d[i] << el;

}
```

下面是一个 cdq 分治优化 dp 的板子（二维LIS，附带统计数量）

```
int n, m;
struct Node {
    int idx, h, v, f; double g;
}a[maxn], tmp[maxn];

struct segtree {
    // 该线段树下标代表 v，值代表当前 v 的最大 f (也就是 mx)，以及取到最大 f 的总方案数 sum
    // 单点更新，区间查询最大值，快速清空
#define ls x<<1
#define rs x<<1|1
#define MID int mid = t[x].l+t[x].r>>1
    struct segNode {
        int l, r, mx, tag;
        double sum;
    }t[maxn * 4];
    int curTag;
    void reset(int x) {
        if (t[x].tag != curTag) {
            t[x].tag = curTag;
            t[x].mx = -inf;
            t[x].sum = 0;
        }
    }
```

```
    void pushup(int x) {
        reset(ls); reset(rs);
        t[x].mx = max(t[ls].mx, t[rs].mx);
        t[x].sum = 0;
        if (t[ls].mx == t[x].mx) t[x].sum += t[ls].sum;
        if (t[rs].mx == t[x].mx) t[x].sum += t[rs].sum;
    }
    void build(int x, int l, int r) {
        t[x] = { l,r,-inf,0,0 };
        if (l == r) return;
        MID;
        build(ls, l, mid);
        build(rs, mid + 1, r);
    }
    void update(int x, int pos, int val, double cnt) {
        reset(x);
        if (t[x].l == t[x].r) {
            if (val > t[x].mx) {
                t[x].mx = val;
                t[x].sum = cnt;
            }
            else if (val == t[x].mx) {
                t[x].sum += cnt;
            }
            return;
        }
        MID;
        if (pos <= mid) update(ls, pos, val, cnt);
        else update(rs, pos, val, cnt);
        pushup(x);
    }
    pair<int, double> query(int x, int ql,int qr) {
        reset(x);
        if (ql <= t[x].l && t[x].r <= qr) {
            return { t[x].mx, t[x].sum };
        }
        MID;
        pair<int, double> L = { -inf, 0 }, R = { -inf, 0 }, res = {-inf,0};
        if (ql <= mid) L = query(ls, ql, qr);
        if (qr > mid) R = query(rs, ql, qr);
        res.first = max(L.first, R.first);
        if (L.first == res.first) res.second += L.second;
        if (R.first == res.first) res.second += R.second;
        return res;
    }
    void clear() {
        curTag++;
    }
#undef ls
#undef rs
#undef MID
}T;

ll f1[maxn], f2[maxn]; // f1: 以 i 结尾的最长下降子序列的长度，f2: 以 i 为开头...
double g1[maxn], g2[maxn];// g: 方案数

// CDQ 分治优化 DP 步骤：
```

```
// 先处理左半
// 统计左半对右半的贡献
// 递归处理右半
void cdq1(int l, int r) {
    if (l == r) {
        T.clear(); return;// 有返回就要清空
    }
    int mid = l + r >> 1;
    cdq1(l, mid); // 处理左半

    sort(a + mid + 1, a + r + 1, [](auto& x, auto& y) {return x.h > y.h; });
    // 需要有序，所以右半先排序
    int j = l, i = mid + 1;
    for (; i <= r; i++) {
        while (j <= mid && a[j].h >= a[i].h) {
            T.update(1, a[j].v, a[j].f, a[j].g);
            j++;
        }
        auto [mx, cnt] = T.query(1, a[i].v, m);
        if (mx + 1 > a[i].f) {
            a[i].f = mx + 1;
            a[i].g = cnt;
        }
        else if (mx + 1 == a[i].f) {
            a[i].g += cnt;
        }
    }
    // 排序，变为初始情况
    sort(a + mid + 1, a + r + 1, [](auto& x, auto& y) {return x.idx < y.idx; });
    cdq1(mid + 1, r);// 递归右半

    // 返回 h 有序数组
    j = l, i = mid + 1;
    int p = l;
    for (; i <= r; i++) {
        while (j <= mid && a[j].h >= a[i].h) {
            tmp[p++] = a[j++];
        }
        tmp[p++] = a[i];
    }
    while (j <= mid) { tmp[p++] = a[j++]; }
    for (int i = l; i <= r; i++) { a[i] = tmp[i]; }
    T.clear();
}
void cdq2(int l, int r) {

    if (l == r) { T.clear(); return; }
    int mid = l + r >> 1;
    cdq2(l, mid);

    sort(a + mid + 1, a + r + 1, [](auto& x, auto& y) {return x.h < y.h; });
    int j = l, i = mid + 1;
    for (; i <= r; i++) {
        while (j <= mid && a[j].h <= a[i].h) {
            T.update(1, a[j].v, a[j].f, a[j].g);
            j++;
        }
```

```
        auto [mx, cnt] = T.query(1, 1, a[i].v);
        if (mx + 1 > a[i].f) {
            a[i].f = mx + 1;
            a[i].g = cnt;
        }
        else if (mx + 1 == a[i].f) {
            a[i].g += cnt;
        }
    }
    sort(a + mid + 1, a + r + 1, [](auto& x, auto& y) {return x.idx > y.idx; });
    cdq2(mid + 1, r);

    j = l, i = mid + 1;
    int p = l;
    for (; i <= r; i++) {
        while (j <= mid && a[j].h <= a[i].h) {
            tmp[p++] = a[j++];
        }
        tmp[p++] = a[i];
    }
    while (j <= mid) { tmp[p++] = a[j++]; }
    for (int i = l; i <= r; i++) { a[i] = tmp[i]; }
    T.clear();

}
void solve() {
    cin >> n;
    vector<int> vec;
    for (int i = 1; i <= n; i++) {
        int h, v; cin >> h >> v;
        a[i] = { i, h,v, 1, 1 };
        vec.push_back(v);
    }
    ranges::sort(vec);
    vec.erase(unique(vec.begin(), vec.end()), vec.end());
    m = vec.size();
    for (int i = 1; i <= n; i++) {
        a[i].v = lower_bound(vec.begin(), vec.end(), a[i].v) - vec.begin() + 1;
    }
    T.build(1, 1, m);

    cdq1(1, n);
    sort(a + 1, a + 1 + n, [](auto& x, auto& y) {return x.idx < y.idx; });
    for (int i = 1; i <= n; i++) {
        f1[i] = a[i].f;
        g1[i] = a[i].g;
        a[i].f = a[i].g = 1;
    }
    // 反转后 f2 变为 1..i 的 LIS
    reverse(a + 1, a + 1 + n);
    cdq2(1, n);
    sort(a + 1, a + 1 + n, [](auto& x, auto& y) {return x.idx < y.idx; });
    for (int i = 1; i <= n; i++) {
        f2[i] = a[i].f;
        g2[i] = a[i].g;
    }
    // 最长下降子序列以及数量
```

```
    ll r = 0;
    for (int i = 1; i <= n; i++) {
        r = max(r, f1[i]);
    }
    cout << r << el;
    double sum = 0;
    for (int i = 1; i <= n; i++) {
        if (f1[i] == r) sum += g1[i];
    }
    for (int i = 1; i <= n; i++) {
        if (f1[i] + f2[i] - 1 == r) {
            // 选择到它的概率
            cout << fixed << setprecision(6)<< g1[i] * g2[i] / sum << " ";
        }
        else cout << 0 << " ";
    }
}
```

杂项

卡常

1. 离线建边，统计每个点的出度，使用 vector 的 resever 提前预留空间防止扩容太慢。

代码审查

盯代码 30 min/ WA 3 发，立刻写对拍

1. 检查数组大小，有时 2 倍也不够；
2. 检查清空大小，$1\sim n$ 够吗，$0\sim n+1$ 够吗，是否所有的数组都得到了清空；
3. 检查各种前缀后缀初始化的范围够不够，$n+1$ 一定够吗；
4. 再读一遍题；样例手动测了吗；
5. 检查 dp 转移对不对；检查一些别的东西转移对不对；
6. 判断条件永远不嫌多，除非你写错了；
7. 线段树/数组下标是编号还是 dfn 序；以及其他的数组下标和你需要的是不是一样的？对于常数也要检查；
8. 检查有没有乘法爆 int，有没有求和爆 longlong；
9. 区间的范围算对了吗；
10. 合并所有有交集的区间时，注意右端点更新应该是 `r = max(r, pr);`，r 应当单调增大；
11. 尝试计算个数时，有没有特判无解直接返回 0，或者有没有取不到的情况，对于组合数函数的负数参数，记得特判；
12. 检查取模有没有取好了，尤其是减法，不好判断请写自动取模类；
13. 检查小的循环次数是否符合预期；
14. 检查 `to_string(x)` 有没有写成 `x+'0'`；
15. 检查标兵数据对不对，0 够吗，INF 够吗，同样还有 mx 和 mi 变量的初始化；
16. 检查 `continue` 和 `return` 有没有把重要的检查跳过了；
17. 每个位置单独统计时，请开全局**数组**，或者局部**变量**；
18. 检查 $\leq$ 和 $<$ 有没有地方不对；
19. 线段树更新到完全覆盖的区间时，是不是该更新的信息都更新了；
20. 检查有没某个分支一部分处理（只处理了特殊点，没有处理普遍点，或者反过来）；